Feedback Is All You Need

Research working draft: version 7

25 October 2026 (UTC)

Abstract

Can a small controller acquire high capability by repeatedly incorporating feedback? We examine restricted forms of this hypothesis in six diagnostic studies. In changing classification tasks, a 161-parameter learner with accurate feedback outperforms a 2,561-parameter frozen learner by 52.91 percentage points on linear rules and 37.32 on XOR rules. When both learn, the larger model leads by 10.93 points on XOR. In hidden-command languages, correct examples provide no measured compositional benefit for the smallest of four pretrained language models under first-token scoring. Full generation reveals stronger oracle execution in 7B but pervasive format or token-cap failures under other conditions. A separately frozen follow-up holds model, instructions, and parser fixed while extending continuations from 128 to 512 tokens. On 24 new languages, clean-example composition accuracy rises from 7.94% to 70.57%, a paired gain of 62.63 points (95% CI [56.10,69.16]); no-example accuracy is 14.06% at the longer budget. All 481 recovered clean composition answers had invalid shorter prefixes. Thus response completion materially changes the measured feedback effect, and short-budget failures cannot establish inability to use examples. A further 24-environment study applies one low-rank parameter update after each verified outcome in the 0.5B model. After 16 outcomes, composition accuracy changes by -0.65 points (95% CI $[-4.04,2.74]$); updating only after errors also yields no demonstrated gain. A sixth study stores generated rules outside the model and compares two self-revisions with two revisions supplied with exact example checks. The primary 0.5B composition difference is $+4.04$ points (95% CI $[-2.78,10.85]$); no small-model proposal recovers a complete rule vector. In 7B, final exact recovery is $6/24$ with checks and $3/24$ with self-review. These findings support conditional adaptation and careful accounting of feedback, capacity, prior learning, and inference resources. They establish neither feedback sufficiency for general intelligence nor a mechanism by which an arbitrarily small agent acquires it.

1 Introduction

The motivating hypothesis is that a compact cognitive core, coupled to an efficient loop of action, outcome observation, correction, and memory, may acquire capabilities disproportionate to its parameter count. Books and other external records suggest an analogy with accumulated human experience. That analogy motivates investigation; it does not establish a mechanism of human intelligence.

The hypothesis must be distinguished from two stronger assertions: that feedback is literally sufficient without representation or inductive bias, and that existing scaling results are wrong. Training already uses feedback through losses, gradients, rewards, and preferences. Scaling-law work measures relationships among loss, capacity, data, and computation [4, 5]; it does not rule out online adaptation. We therefore compare capacity and feedback as interacting resources.

We retain the researcher's chosen title and follow the organizational pattern of *Attention Is All You Need* [3]: background, mechanism, motivation, experimental specification, results, and conclusion. The title is a hypothesis under examination, not an established sufficiency theorem.

Chen et al. already used a closely related title in 2023 [6]; we make no priority claim for the phrase or broad feedback-centered framing.

2 Background

The broad motivating idea has a direct precedent in *Reward is enough* [2]. Silver and colleagues hypothesize that abilities associated with intelligence can arise in service of reward maximization in a rich environment. Their account explicitly includes bounded capacity and real-time computation, and provides no sample-efficiency guarantee. Our experiments examine particular learners and feedback mechanisms; they do not test every agent covered by that conjecture.

The later *Welcome to the Era of Experience* perspective [1] directly advocates sustained interaction, rewards grounded in consequences, continuing adaptation, and revised world models. It also recognizes joint scaling with network size, experience, and thinking time. The broad agenda therefore precedes this work; our short synthetic studies do not instantiate its proposed lifelong learning system.

Fast adaptation in recurrent state was studied in RL^2 [7] and in-context reinforcement learning in Algorithm Distillation [8]. RL^2 separates a slowly learned update algorithm from fast changes in hidden state. A small runtime controller can therefore embody expensive prior learning. Reflexion combines an evaluator, verbal reflection, and episodic memory [9]; its feedback sources vary across tasks. Self-Refine uses the same model to generate, critique, and revise an answer with task-specific prompts [10], and explicitly requires sufficient instruction-following ability. Our sequential guess condition reproduces neither method.

Huang et al. distinguish intrinsic self-correction from correction supplied with external correctness signals [11]. Their evaluation also stresses inference-cost comparisons and equally informative initial instructions. Test-time compute studies show that some smaller models can outperform larger ones under specific inference allocations [12]. Accordingly, a small-versus-large win alone is not a new account of intelligence. Our current contribution is a reproducible diagnostic comparison of feedback, capacity, and response interfaces, including failed mechanisms and explicit controls. The literature review is targeted rather than exhaustive; exact versions and reading scope are recorded with the repository's source manifests.

Executable and abstract memory have close precedents. Voyager combines Minecraft feedback, execution errors, self-verification, and an executable skill library with GPT-4/GPT-3.5 components and a separate embedding model [13]. Self-Debugging combines execution/test results with generated explanations or simulated traces [14]; its traces are not observed intermediate states, and Spider still supplies executed SQL results without a unit-test correctness oracle. ExpeL appends extracted natural-language insights and retrieves successful trajectories [15]. Its HotpotQA-to- FEVER transfer adapts insights using GPT-4 and target-task examples, without experience-pool retrieval in that transfer setting. These systems precede our external-memory proposal. They motivate controlled tests of verification and representation, not a novelty claim for agent memory itself.

3 Agent and Feedback Model

An agent has parameters θ, persistent memory m, and an interaction history h. Its policy selects an action, and an update U incorporates an environmental response f:

$$a_t \sim \pi_{\theta_t}(\cdot \mid o_t, m_t, h_t), \qquad (\theta_{t+1}, m_{t+1}, h_{t+1}) = U(\theta_t, m_t, h_t, o_t, a_t, f_t).$$

An implementation may update parameters, memory, context, or some combination. Its entire system capacity includes external storage and algorithms; a small neural core is not necessarily a small complete system.

Study A instantiates this notation with a binary classifier whose action is a label prediction. Feedback gives the true label after prediction. Inputs are independent of the action, and gradient descent updates the parameters. This is supervised online learning, not active exploration or a new reinforcement learning algorithm. In a binary task, correctness plus the chosen label also determines the target label.

4 What Feedback Can and Cannot Supply

4.1 An information constraint

Let Θ be uniform over $K > 1$ tasks. Assume the complete initial agent state and all side information are independent of Θ. Count every subsequent task-dependent observation as part of the response transcript. If each of n responses has an alphabet of size at most 2^b, then $I(\Theta; F_{1:n} \mid S_0) \leq nb$. Adaptive queries are functions of S_0 and preceding responses, so cannot independently supply missing task information. Fano's inequality gives the exact-task identification bound

$$P_e \geq \max\left\{0, 1 - \frac{nb+1}{\log_2 K}\right\}.$$

Indeed, $H(\Theta \mid F_{1:n}, S_0) \leq 1 + P_e \log_2 K$; subtracting this entropy from $\log_2 K$ yields the result. The bound is about task identification, not every prediction or reward metric. Informative priors or unaccounted observation channels invalidate these assumptions.

For a binary symmetric channel with independent flip probability p, the per-response information is at most $1 - H_2(p)$ bits. At $p = 1/2$, responses carry no information about the true bit. At $p = 1$, inversion is recoverable if the corruption mechanism is known. Learner failure under inverted labels therefore diagnoses a misspecified channel, not destroyed information.

4.2 Representation and transcript constraints

If a successful policy is outside the complete agent's permitted policy class, additional updates within that class cannot express it. External memory or tools may enlarge that class, so this constraint cannot be inferred from neural parameter count alone. In addition, a deterministic updater replaying the same ordered transcript from the same initial state reaches the same final state, by induction. Calling the procedure a loop adds no independent advantage. Active information acquisition or changing observations can matter; these mechanisms require their own tests.

Under the same independent-prior assumption, suppose the final retained state has at most 2^M possible values, with M counting all retained task-dependent bits. Without further task-dependent observations, identification of a uniform task among K possibilities has success probability at most $\min\{1, 2^M/K\}$. For any encoder, the optimal decoder succeeds with probability $K^{-1} \sum_s \max_k P(s \mid \Theta = k) \leq 2^M/K$. This bound concerns exact identification, not every prediction metric. External records belong in the retained state; their capacity cannot be omitted simply because neural parameter count is small. It yields no universal threshold in numbers of neural parameters.

5 Experimental Setup

5.1 Tasks, models, and feedback

Each seed defines an initial and an independently generated post-change rule over eight standard-normal inputs. Linear labels threshold a normalized random projection. XOR labels combine signs of two randomly selected coordinates, with an independent fair complement bit for each rule. Warmup comprises 256 batches of 32 examples; adaptation uses a further 256 batches from the new rule. Evaluation sets of 2,048 examples per rule never enter updates. Separate uniform-input tests preserve each rule and change its input distribution.

The models are logistic regression and one-hidden-layer ReLU networks of widths 16, 64, and 256: 9, 161, 641, and 2,561 parameters. Adam uses learning rate 0.01 and default moments, with fresh moments after warmup. Every arm at the same capacity and seed starts from identical warmed-up parameters and sees the same arriving input stream.

The seven arms are frozen weights, clean labels, 25% flips, 50% flips, inverted labels, clean feedback delayed by 32 batches, and 25% replay. Replay replaces the final eight new examples in every batch with eight warmup examples sampled with replacement, retaining their original labels. It therefore changes both memory use and fresh-example exposure. Delayed feedback has 224 updates at arrival batch 256; a separate flush processes the remaining 32 without new observations.

5.2 Protocol, statistics, and resources

The full grid uses 20 seeds, 4000–4019, and was frozen in repository history before collection. A development run accidentally exposed seeds 1000 and 1001; the seed set was amended before formal collection, without hyperparameter tuning. A subsequent code audit found overlapping PRNG streams in the first full run (seeds 2000–2019). That run is archived with a superseded-inference notice. This version reports a replication on untouched seeds 4000–4019, with streams derived by SHA-256 of environment, family, role, and step or width. A complete-grid collision check passed. The protocol, hyperparameters, and endpoints otherwise remain unchanged. This is repository registration, not external preregistration.

The primary contrast in each family compares width-16 clean to width-256 frozen at arrival batch 256. This deliberately unequal-information comparison tests adaptation, not compute superiority. We report paired seed-level t intervals at 97.5% for each of these two contrasts (Bonferroni familywise 95%). Secondary 95% intervals are descriptive. Seeds, not checkpoints or individual examples, are the replication units; t inference assumes adequate approximation of the seed-mean sampling distribution.

The run used PyTorch 2.8.0 with CUDA 12.6 on one RTX 4090 D and completed 1,120 arms in 109.54 seconds. Code and environment hashes, all 19,200 checkpoint rows, and analysis are retained. For arithmetic-budget comparisons we count $6WN$ multiply-add-related training FLOPs for W matrix weights and N examples. This proxy excludes optimizer, activation, bias, and hardware overhead. Warmup and adaptation are reported separately; it is not lifetime compute or a measured energy budget.

5.3 Study B: pretrained language models

We compare Qwen2.5-Instruct models with 0.494, 1.544, 3.086, and 7.616 billion parameters. Exact Hugging Face revisions and package versions are archived. Each of 24 environments (seeds 3000–3023) assigns four named commands distinct hidden offsets modulo eight. A command adds its offset to an integer state; compositions apply two commands. The family is disclosed, while

each environment's offsets are withheld except in an oracle control. Train/test inputs are disjoint: 16 single-command feedback examples, 16 held-out single commands, and 32 held-out length-two compositions per environment. These translations commute, so the test does not measure command-order sensitivity.

Frozen-model conditions are no examples, 4/8/16 correct examples, 16 corrupted examples, 16 sequential model guesses, and explicit oracle offsets. Corruption replaces an answer with a uniformly chosen wrong digit with probability 0.5; this channel still conveys information. Self-generated examples use each model's constrained predictions conditioned only on its earlier guesses. All histories share the same demonstration template. Query selection is fixed, so this is passive adaptation rather than action-dependent exploration. Giving the final correct transcript offline produces byte-identical prompts; we do not count duplicate calls as independent evidence of an online effect. A hand-coded modular-offset estimator solves every test after the same clean examples, demonstrating identifiability within the supplied algorithmic family.

We select the most likely first assistant token among digits 0–7 in bfloat16, without sampling or additional reasoning tokens. Each digit is one tokenizer token. We also record unrestricted first-token accuracy, which does not measure full-response validity. This narrow interface can underuse a model's generative reasoning ability and is a central limitation, not an intelligence metric.

For the 0.5B model, rank-8 LoRA updates all query/value projection matrices (alpha 16, zero dropout), using 16 example-free training prompts for 32 passes, batches of four, AdamW at 2×10^{-4}, and a next-digit cross-entropy loss. Each environment starts from the same base checkpoint and fresh adapters and optimizer. Tests establish exact base-weight equality after two adapter train/unload cycles. Held-out tests use either no examples or the full correct context. There is no checkpoint selection. This measures persistence on one language, not lifelong retention across unrelated tasks.

The primary Study B endpoint is the paired 0.5B clean-16 minus self-generated accuracy on compositions, across 24 environments, with a 95% t interval and 10,000-resample paired bootstrap sensitivity interval. All other intervals are descriptive. Comparing released model sizes confounds parameter count with their different pretraining histories; no causal scaling-law estimate is claimed. All model calls run locally on the same RTX 4090 D, with full query predictions, prompt hashes, token counts, measured times, and peak memory.

5.4 Study C: response-interface sensitivity

We freeze 24 additional environments (seeds 5000–5023) before formal inference and evaluate the same pinned 0.5B and 7B checkpoints. Each environment has the same 16 feedback, 16 direct, and 32 composition queries as Study B's design. The four conditions are no examples, 16 correct examples, 16 sequential guesses, and oracle offsets. No weights change in this study.

Each condition is evaluated with Study B's constrained first-token interface and with full generation. In the latter, both digit-only instructions are replaced by a request for brief reasoning followed by a final line `Answer: N`. Decoding is greedy with a 128-new-token cap and inherited checkpoint defaults: repetition penalties are 1.1 for 0.5B and 1.05 for 7B. After tokenizer special tokens are removed, the last nonempty line must match `Answer: [0-7]`, ignoring case and optional whitespace. Invalid formats count as wrong. Self-history stores the parsed digit or the literal string `invalid`. All output tokens through EOS, decoded text, invalid formats, truncations, input/output tokens, and measured time are retained.

The primary endpoint is the paired environment-level interaction for 0.5B composition accuracy,

$$\Delta_{\text{int}} = (A_{\text{clean}} - A_{\text{none}})_{\text{generation}} - (A_{\text{clean}} - A_{\text{none}})_{\text{first token}},$$

with a 95% t interval and a 10,000-resample bootstrap interval. Remaining intervals are descriptive.

An interaction must be interpreted with its component means and within-interface feedback effects: it can be positive because the control deteriorates. The intervention changes instructions, output constraints, and compute jointly; it does not isolate the effect of reasoning tokens. Development uses seed 9 only. Prompt and cap remain fixed after its failures. A prose error that described both repetition penalties as 1.1 was corrected after launch; the frozen rule to inherit model defaults and the runner were unchanged. Exact source hashes and this amendment are retained.

5.5 Study D: longer generation with fixed instructions

Study C's truncation motivates a separately frozen follow-up on 24 new languages (seeds 6000–6023). Only the pinned 7B model is evaluated, with unchanged instructions, weights, parser, batch size, and generation defaults. The conditions are no examples, clean16, and oracle offsets. Each query has one greedy continuation capped at 512 new tokens. We score both its first 128 tokens and its full continuation. Under this configuration, decoding before the shorter cap is independent of the cap: there is no length-dependent forced EOS, beam search, or minimum-length control. On development seed 11, all 144 prefixes exactly match separately generated 128-token responses. Source hashes, model identity, and settings match the formal runner; every compared pair is retained before formal collection.

The primary endpoint is the environment-paired clean16 composition difference, $A_{512} - A_{128}$, with a 95% t interval and 10,000-resample bootstrap. Other conditions, direct accuracy, feedback effects, format rates, truncation, and correct-to-wrong transitions are descriptive. No monotonic improvement is assumed: later text can revise an answer or invalidate the required final line. There are 1,152 environment/query pairs evaluated in three conditions, giving 3,456 actual continuations in 216 batches. Prefixes do not count as independent replicates or additional model calls, and shorter-budget wall times are not invented. The 0.5B model is excluded because its Study C responses terminated at EOS after two tokens; a higher cap cannot extend those completed responses under identical instructions. This selection is explicit and does not test alternative small-model prompts.

5.6 Study E: online outcomes and parameter updates

To test repeated updates explicitly, we freeze 24 further environments (seeds 7000–7023) with the same adaptation, direct, and composition splits. The pinned 0.5B model uses Study B's constrained first-token scoring. Three conditions receive the same ordered 16 adaptation queries. The context condition predicts, observes the true target, and appends the correct pair. The per-outcome LoRA condition predicts without examples, observes the target, and applies one gradient update. The error-only condition applies that update only after an incorrect constrained prediction. Targets enter no prediction prompt, and every pre-update prediction is flushed to an append-only journal before optimization. Feedback inputs are fixed, not actively selected.

LoRA uses the same rank, modules, optimizer, and learning rate as Study B, but one update on the current example per outcome: at most 16 singleton updates. Both adapter conditions start from identical adapter tensors for each environment. All weights and optimizer state reset between conditions and environments. Evaluations after 0, 1, 2, 4, 8, and 16 outcomes use held-out queries; adapters are evaluated without examples, with an additional full- context evaluation only at the final checkpoint. No best checkpoint is chosen. These 16 updates are a smaller optimization dose than Study B's 128 updates.

The primary endpoint is environment-paired per-outcome LoRA composition accuracy at 16 outcomes minus the frozen no-feedback baseline, with a 95% t interval and a 10,000-resample bootstrap (seed 821). Remaining contrasts and learning-curve intervals are descriptive. The error-only

arm can use fewer updates; its compute is measured rather than matched. An ordered offline replay of the per-outcome labels and updates would be equivalent. We test this update rule's retained within-environment performance, not an independent benefit of the online presentation or retention after learning another task. The restricted interface also leaves generative ability unmeasured.

Development seed 13 passes the complete audit before formal collection. Exact base-parameter hashes match after each development condition and before and after the formal suite. Zero-feedback predictions agree with the base in every environment. Raw evaluation records, ordered prediction/feedback pairs, all 48 final adapters, provenance, and measured costs are retained.

5.7 Study F: checked revision of executable rule memory

We freeze 24 new languages (seeds 8000–8023), with development seed 17, and use the same pinned 0.5B and 7B models. Each model sees the additive family and all 16 single-command examples, and outputs four distinct integer offsets as a JSON object. The exact four command keys and values 0–7 are required; duplicate keys, invalid types, or surrounding prose invalidate the proposal. A single outer code fence is accepted. A supplied modular-addition executor uses only the saved rule to answer the 16 direct and 32 composition queries. No generated code is executed. Neither held-out inputs nor targets enter proposal or revision prompts.

One initial proposal is shared by two branches. Each branch receives exactly two further greedy calls, capped at 512 new tokens each. Self-review repeats the examples and the immediately previous proposal and requests a revision. Checked review adds a deterministic check: format failure, all examples pass, or the first failing example with the proposal's output and the already-known correct output. This supplies computed verification, not new task information or the true offset vector. Both branches run both revisions even after a passing proposal; the final proposal is scored without selecting a better earlier candidate. We retain every intermediate memory and response.

The primary endpoint is 0.5B final checked-minus-self composition accuracy, paired across environments. A positive primary effect requires the 95% t interval's lower bound above zero; a 10,000-resample percentile bootstrap (seed 822) is a sensitivity check. Each branch's final-minus-initial effect is reported to distinguish improvement from degradation of the control. Exact-rule recovery, direct accuracy, validity, all stages, and 7B are secondary. Binary environment rates use Wilson 95% intervals; query accuracy and transition proportions use t intervals over environment means. In particular, adding four modulo eight to every true offset preserves all two-command outputs while making every direct answer wrong. Perfect composition therefore does not establish correct rule memory.

All examples are available initially. An exact symbolic estimator provides an identifiability control. There is no active exploration, weight update, cross-language accumulation, or test of neural retention after examples are removed. The rule file persists outside the model, and the executor supplies execution competence. Equal call caps are not equal compute: prompt lengths and stopping times differ. Token counts, padding, generation time, checker time, and memory bytes are retained, with the shared initial generation counted once. Only a matching audited development run permits formal inference.

6 Results

6.1 Adaptation and capacity

The small adaptive model exceeds the large frozen model by 52.91 points on linear tasks (97.5% CI [47.62,58.20]) and 37.32 on XOR ([28.85,45.79]). When both receive feedback, small-minus-

Table 1: Mean post-change held-out accuracy (%), 20 seeds. All adaptive columns receive the same 8,192 clean examples; frozen models receive none.

Parameters	Linear frozen	Linear clean	XOR frozen	XOR clean
9	45.61	90.67	50.12	49.87
161	45.56	98.53	47.64	84.97
641	45.64	98.71	47.69	96.14
2,561	45.62	98.53	47.65	95.90

large is 0.00 points for linear (95% CI [-0.22,0.22]) and -10.93 for XOR ([-15.39,-6.46]). The latter comparison shows a substantial remaining capacity effect under the tested update schedule. Logistic regression remains near chance on XOR despite clean feedback. Exact XOR is outside this fixed linear policy class, but observed chance performance also depends on the loss and optimization: a biased linear threshold can exceed chance under a zero-one objective. We do not identify chance as the best attainable linear-classifier accuracy or a limit for all small controllers.

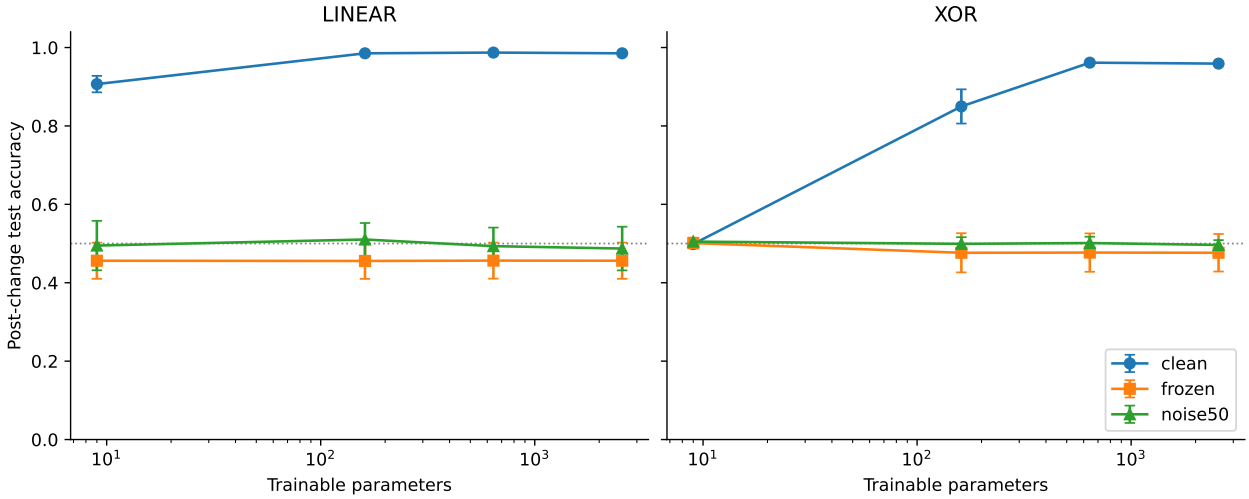


Figure 1: Capacity and feedback jointly determine observed accuracy. Error bars are descriptive 95% t intervals across 20 seeds. The dotted line is chance accuracy; noisy feedback can coexist with information retained from warmup, so individual noisy outcomes need not equal chance.

At an equal adaptation arithmetic proxy, the small model's 256 updates and the largest model's 16 updates both cost 7,077,888 proxy FLOPs. The small model leads by 42.80 points on linear tasks and 34.72 on XOR. However, it receives sixteen times as many new examples, while the large model's warmup cost is sixteen times greater. This is a particular compute/information trade, not an isolated causal effect of feedback-loop efficiency or a scaling law.

6.2 Feedback variations, transfer, and retention

For the 161-parameter model, clean, 25% corruption, and 50% corruption yield 98.53%, 92.05%, and 51.03% accuracy on linear tasks and 84.97%, 71.84%, and 49.92% on XOR. Inverted feedback yields 1.20% and 13.32% respectively. More updates can reinforce the wrong target when the update rule assumes trusted labels.

Delayed feedback reaches 98.28% and 81.87% at the online endpoint. After flushing, its scores and final training losses equal clean feedback for every seed and capacity. This equality is a correctness check for ordered updates, not an additional empirical discovery about latency.

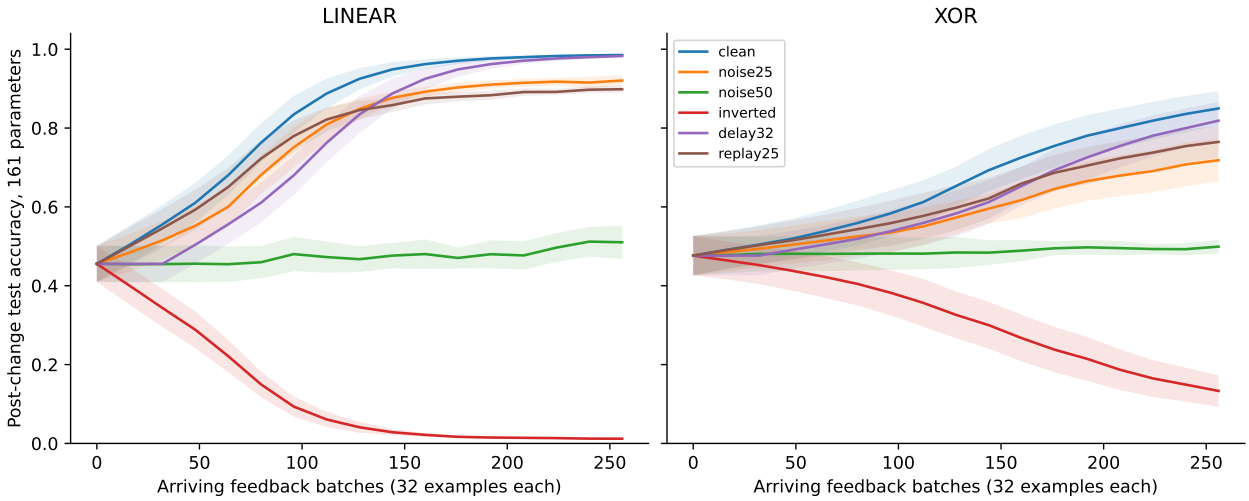


Figure 2: The same 161-parameter learner under six update conditions. Shading shows descriptive 95% seed-level intervals at fixed arrival checkpoints.

Clean small-model uniform-input accuracy is 98.44% for linear rules and 88.19% for XOR. This is distribution transfer within the same known rule family, not transfer to an unrelated task. Old-rule accuracy after clean adaptation is 45.23% and 51.94%. Replay raises these to 54.92% and 64.43% while lowering new-rule accuracy to 89.85% and 76.48%. Because rules may conflict without a task identifier, this is an adaptation– retention tradeoff; perfect performance on both is not always possible.

6.3 Study B: feedback does not guarantee compositional gains

Table 2: Study B mean constrained first-token accuracy (%), 24 environments. Direct tests use one command; composition tests use two. Chance is 12.5%.

Model	Direct none	Direct clean	Compose none	Compose clean	Compose oracle
0.5B	13.80	12.24	11.46	11.46	11.46
1.5B	13.28	13.02	11.20	12.63	12.89
3B	10.16	5.73	13.02	12.63	14.32
7B	13.80	26.56	13.80	17.58	26.82

For the primary 0.5B comparison, clean feedback and self-generated examples both yield 11.46% composition accuracy. The paired difference is 0.00 points (95% t CI [-1.03,1.03]; bootstrap [-0.78,1.04]). Clean-minus-none is also 0.00 points. This is a failure to demonstrate the hypothesized gain with this mechanism, not proof that every feedback mechanism must fail.

The 7B model reaches 26.56% on direct tasks with clean-16 examples versus 13.80% without examples. Its composition scores are 17.58% and 13.80%, respectively: a descriptive paired difference of 3.78 points (95% CI [0.38,7.17]). With explicit offsets, direct accuracy rises to 75.52%, while

composition reaches 26.82%. Representation of the instruction, inference of the latent rule, and execution of compositions are distinct bottlenecks.

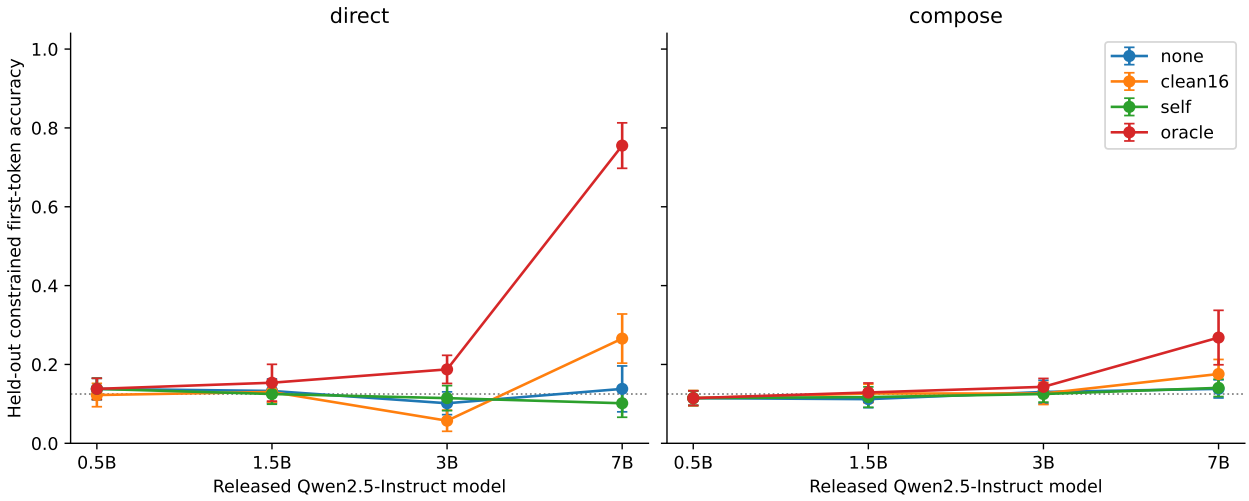


Figure 3: Released-model size and the supplied evidence affect direct and compositional performance differently. Error bars are descriptive environment- level 95% t intervals. The oracle provides extra structured information; it is not a cost-matched competitor.

LoRA trains 540,672 parameters in addition to the frozen 0.494B base. Without example context, its direct accuracy is 26.30% versus 13.80% for the base. The paired direct gain is 12.50 points (descriptive 95% t CI [3.76,21.24]). Composition accuracy is 13.15%; its paired improvement over no-feedback base is 1.69 points (95% CI [-2.12,5.50]). Retaining examples at inference yields 13.28% composition accuracy but reduces direct accuracy to 9.90%. Thus the persistent single-command gain is strongly context-dependent: adding the same demonstrations does not preserve it. These results distinguish a gain under the training prompt format from robust compositional transfer.

Each frozen-model suite processes 2,156,304 unpadded input tokens in 888 forward calls, including self-history construction and evaluation. Measured suite times are 21.81, 56.22, 109.02, and 234.69 seconds; peak allocated memory is 1.20, 3.41, 6.41, and 15.29 GiB. These figures are whole-suite accounting, not equal lifetime budgets or per-condition causal estimates. The LoRA suite adds 1,486,848 training input tokens and 86.67 measured update seconds. Its extra optimization is explicitly not cost-matched to frozen inference.

6.4 Study C: format and budget limit the generation comparison

All 9,216 held-out predictions and 768 self-history records per model pass source, model-revision, environment, completeness, raw-token decoding, and score checks. Table 3 reports composition accuracy; Figure 4 also shows direct predictions.

The primary 0.5B interaction is −0.52 percentage points (95% t interval [−1.16,0.11]; bootstrap [−1.17,0.00]). Its first-token clean-minus-none effect is 0.52 points, while the generation effect is exactly zero in this sample. Every generated answer is the literal `N` followed by EOS, including the oracle condition. This is a failure of the tested instructions and output interface; it cannot establish that the model lacks the reasoning capacity measured by a different interface.

For 7B, oracle generation reaches 100% direct and 97.27% composition accuracy, compared with 69.53% and 24.74% under first-token scoring. Thus the narrow first-token interface can substantially

Model	Interface	No examples	Clean 16	Self	Oracle
0.5B	First token	10.68	11.20	10.68	10.68
0.5B	Generation	0.00	0.00	0.00	0.00
7B	First token	13.15	16.15	12.24	24.74
7B	Generation	0.00	6.90	0.26	97.27

Table 3: Study C composition accuracy (%). Generation scores include invalid and truncated responses as defined by the frozen parser; they must be read with the format and budget results below.

understate execution capability when the true rules are supplied. With examples, generation's clean-minus-none composition contrast is 6.90 points (descriptive 95% interval [4.04,9.76]), but absolute clean accuracy falls from 16.15% to 6.90%. Generation improves that contrast partly against a control with zero measured accuracy; it does not establish a superior overall inference method.

The 128-token cap is material. In 7B composition queries, truncation rates are 100%, 88.28%, 95.70%, and 0.26% for no examples, clean examples, self examples, and oracle offsets respectively. Corresponding invalid-format rates are 100%, 88.28%, 95.57%, and 0.26%. A truncated response can still contain a parseable final line, so these quantities need not coincide. Clean-example direct generation scores 24.22%, with 57.55% truncation. These outcomes leave longer-budget example induction unresolved; they are not evidence of an unrestricted inability to infer the offsets.

Self-history also deteriorates under the interface: all 384 generated examples for 0.5B and 354 of 384 for 7B are invalid. The resulting histories contain the literal `invalid`, as specified. They are not 16 valid critiques or error-directed repairs. The contrast measures the complete construction of this self-history under the fixed prompt and budget.

Whole-suite elapsed times, excluding loading, are 28.92 and 1,831.46 seconds. Each model uses 672 calls per interface, including history acquisition. The 7B first-token and generation paths take 133.83 and 1,695.16 measured forward seconds; generation emits 554,021 tokens, or 599,945 token slots with batch padding. Such costs are recorded, not matched. All generation defaults, including the model-specific repetition penalties, remain part of the treatment.

6.5 Study D: longer continuations recover example-based performance

All 3,456 continuations and 216 actual generation batches pass the registered integrity checks. Table 4 and Figure 5 compare paired prefixes on the same 24 new environments.

	Direct		Composition	
Condition	128 tokens	512 tokens	128 tokens	512 tokens
No examples	3.12	11.98	0.00	14.06
Clean 16	20.05	59.90	7.94	70.57
Oracle offsets	100.00	100.00	96.61	96.88

Table 4: Study D accuracy (%) with fixed instructions and model. Shorter scores use prefixes of the same deterministic continuations.

The primary clean composition difference is 62.63 percentage points (95% t interval [56.10,69.16]; bootstrap [56.38,68.75]). The clean-minus-none effect at 512 tokens is 56.51 points (descriptive 95% interval [48.99,64.03]), compared with 7.94 at 128 tokens. The control also improves as its answers become complete. Thus the larger budget reveals a substantial measured benefit from the correct examples under this fixed interface.

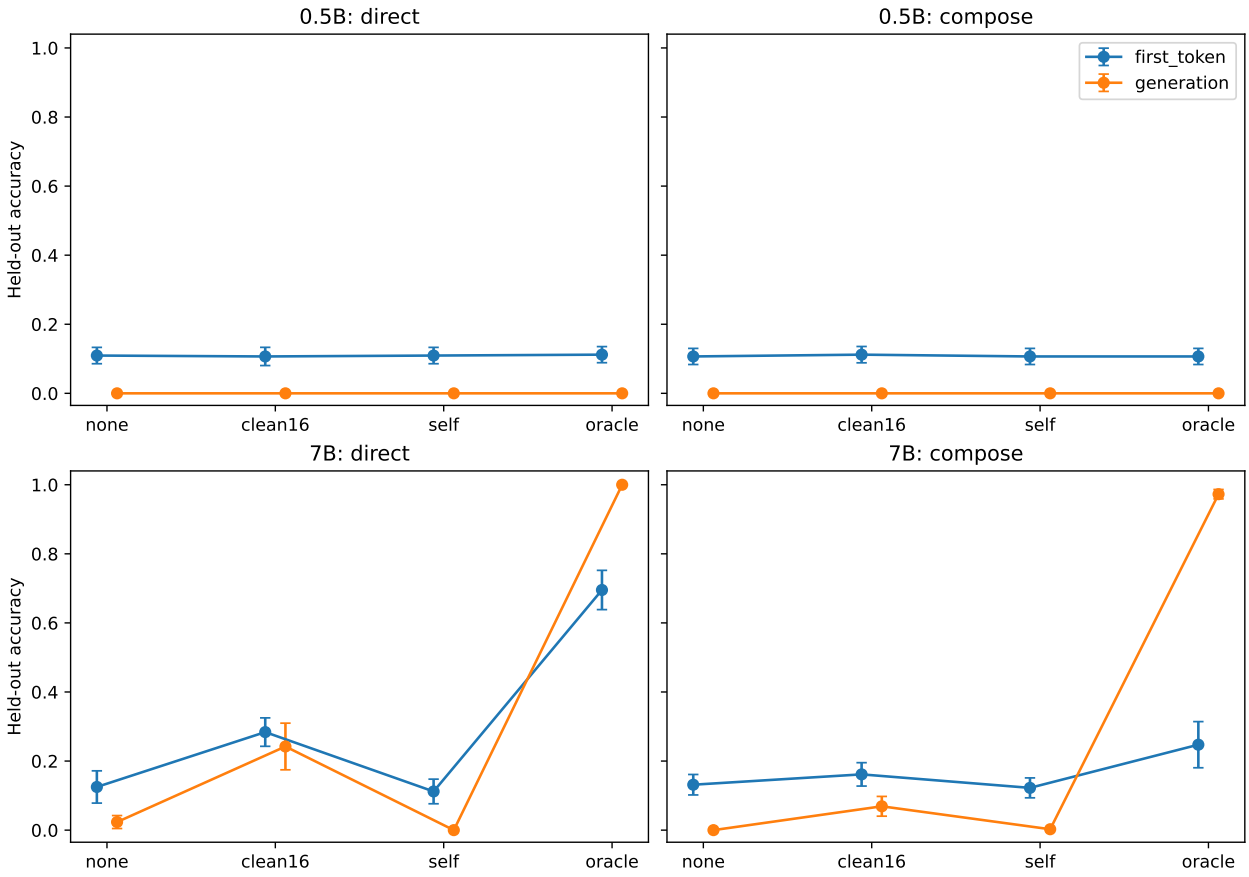


Figure 4: Study C response interfaces. Error bars are descriptive 95% intervals across 24 environment means. Strong oracle execution under generation coexists with pervasive format or token-cap failures in other conditions.

Clean composition truncation falls from 87.76% to 3.26%, and invalid-format rate falls from 87.76% to 2.99%. Among 768 clean composition queries, 481 change from wrong to correct and none change in the opposite direction. All 481 gains occur where the shorter prefix was invalid. Accordingly, this result does not show correction of 481 previously valid wrong answers or isolate a reasoning mechanism from completion of the requested answer format. It shows that early evaluation of an unfinished response can seriously understate the model's use of existing information. No additional environmental feedback arrives during these continuations.

Oracle composition changes little, from 96.61% to 96.88%, while clean direct accuracy rises from 20.05% to 59.90%. These separate direct and composition query sets have different input lengths and generated traces; their ordering is not a general statement that composition is easier than direct prediction. The experiment adds no persistence across tasks, active exploration, or new parameter updates. It neither resolves small-model format failures nor establishes an advantage of a small learner over a large one.

The run takes 1,436.61 seconds, including 1,435.81 measured generation seconds, processing 853,632 input tokens and 654,495 output tokens. Output padding raises token slots to 1,044,320. The shorter prefixes have no separately measured wall times, so these records do not support a 128-versus-512 speed ratio.

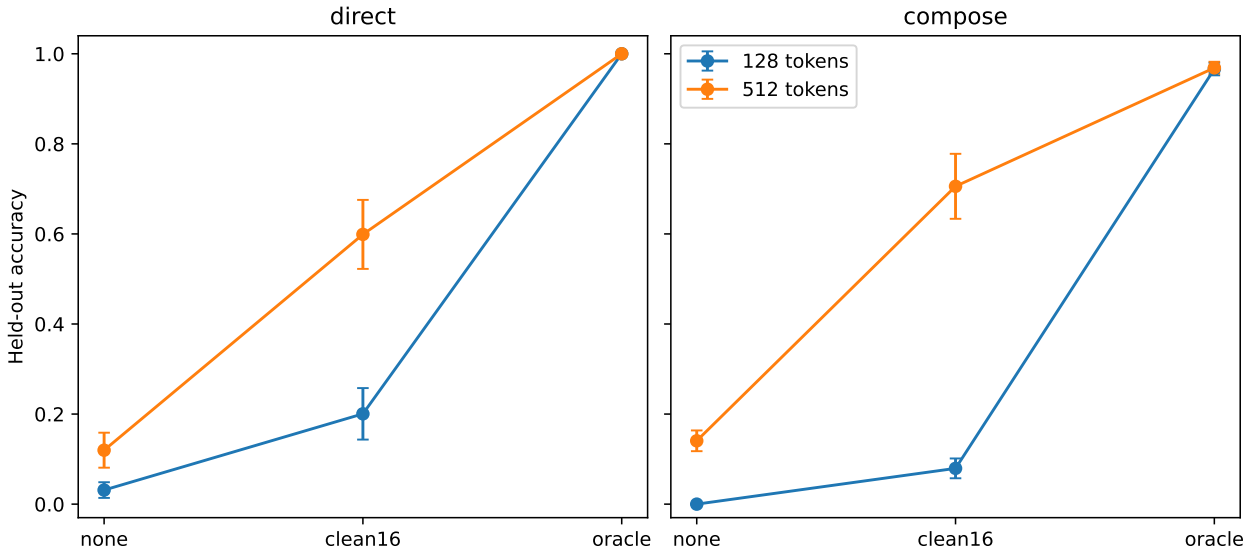


Figure 5: Study D generation budgets. Error bars are descriptive 95% intervals across 24 environment means. Extending the same continuation substantially changes example-based accuracy; oracle execution is already nearly complete at the shorter budget.

6.6 Study E: one-pass updates do not establish retained gains

The complete 24-environment run passes all grid, source, model, prompt, feedback-order, state-reset, and adapter-file checks. Table 5 shows final accuracy; Figure 6 reports the fixed checkpoint curves.

The primary per-outcome composition change is −0.65 percentage points. Its 95% t interval is [−4.04, 2.74]; the bootstrap interval is [−3.78, 2.47]. Of 768 composition queries, 99 change from wrong to correct and 104 change from correct to wrong. The updates change behavior without demonstrating a net gain. Error-only updating has a descriptive change of −2.34 points (95% in-

Condition	Direct	Composition
No feedback	14.06	14.32
Context after 16 outcomes	12.50	14.45
Per-outcome LoRA	13.28	13.67
Error-only LoRA	10.68	11.98
Per-outcome LoRA with context	5.47	13.54
Error-only LoRA with context	5.99	12.89

Table 5: Study E constrained first-token accuracy (%). Adapter-only rows remove the correct transcript at evaluation. Chance is 12.5%.

terval [−5.53,0.85]), with 86 improvements and 104 regressions. Context alone changes composition accuracy by 0.13 points ([−0.34,0.60]). These intervals describe uncertainty under this protocol; a null result is not a proof of no possible feedback benefit.

Adding the correct transcript to the per-outcome adapter lowers direct accuracy from 13.28% to 5.47%, a descriptive paired difference of −7.81 points (95% interval [−12.16,−3.46]). The error-only adapter falls from 10.68% to 5.99% (−4.69 points; [−8.67,−0.70]). This repeats the qualitative context dependence seen in Study B; it does not establish that context is generally harmful. Neither tested update rule yields a demonstrated composition improvement under this short adaptation schedule and interface.

The suite takes 72.80 seconds after model loading, recording 24,192 evaluation rows in 1,512 batches and 1,152 pre-update prediction calls. Per-outcome LoRA applies 384 updates across environments, processing 46,464 optimization input tokens in 7.82 measured seconds. Error-only LoRA applies 337 updates with 40,777 tokens in 6.82 seconds. Whole-suite peak allocation is 1,305,673,728 bytes. Full per-condition inference, padding, and update costs are archived. These are unequal adaptation costs, not lifetime compute estimates.

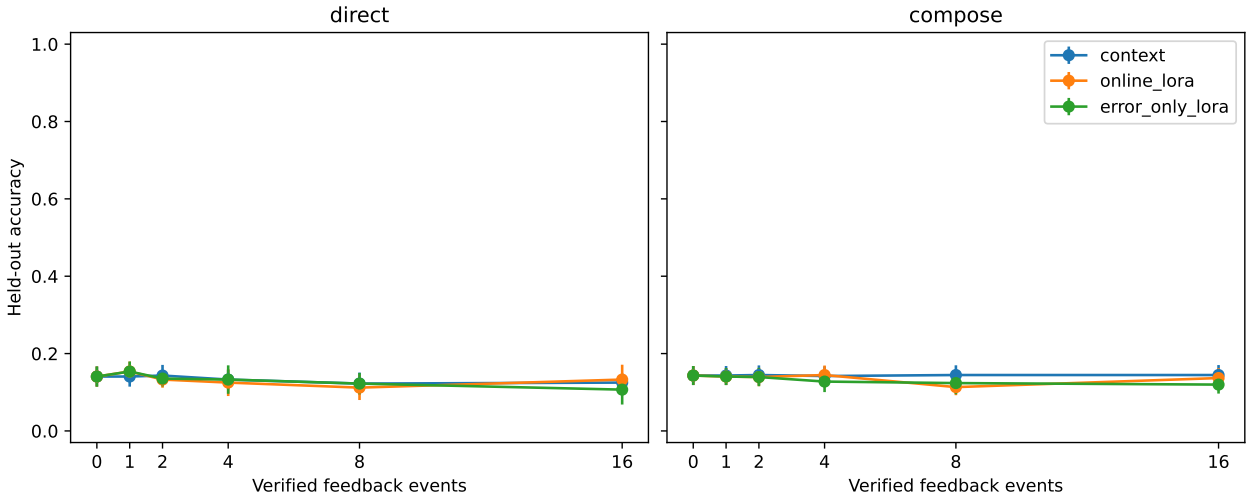


Figure 6: Study E fixed checkpoints. Error bars are descriptive 95% t intervals across 24 environments. Repeated verified outcomes do not produce demonstrated final compositional gains with either tested adapter update rule.

6.7 Study F: external checks do not establish a reliable gain

Both model suites pass the full audit of 120 proposals, 120 rule files, and 5,760 held-out evaluations each. Table 6 reports the initial and final proposals; Figure 7 includes both fixed revisions. Every proposal terminates at EOS before the 512-token cap.

Model	Proposal	Direct	Composition	Valid/24	Exact/24
0.5B	Initial	11.46	9.38	18	0
	Self-review final	11.46	9.38	18	0
	Checked final	10.42	13.41	17	0
7B	Initial	51.04	43.23	21	4
	Self-review final	52.08	42.32	22	3
	Checked final	54.17	45.44	21	6

Table 6: Study F executor accuracy (%) and rule counts. Invalid rules score zero. The stored rules supply offsets; a human-provided algorithm executes the held-out queries. Results measure the combined system.

The primary 0.5B checked-minus-self composition effect is +4.04 percentage points (95% t CI [−2.78,10.85]; bootstrap [−1.56,11.07]). Neither interval demonstrates a positive effect under this protocol. Checked-minus-initial has the same mean and t interval. Self-review has zero net composition change, with two query answers gained and two lost. Against self-review, checked revision gains 60 composition answers and loses 29, while direct accuracy falls by 1.04 points (CI [−10.11,8.02]). No 0.5B proposal recovers all four offsets at any stage; the final 0/24 exact-recovery rate has Wilson 95% CI [0.00%,13.80%]. Thus lack of observed recovery is not a universal zero-rate claim.

For 7B, comparing checked review with self-review gives a composition difference of +3.13 points (CI [−9.32,15.57]). The change from the initial proposal is +2.21 points (CI [−9.93,14.36]). Checked revision recovers two previously incorrect rule vectors and retains all four initially correct vectors. Self-review loses one initially correct vector and gains none. Final exact recovery is 6/24 (25.00%, Wilson CI [12.00%,44.90%]) versus 3/24 (12.50%, CI [4.34%,31.00%]); these are descriptive rates, not a confirmed between-arm effect. Against the initial proposal, checked review gains 81 composition answers and loses 64. Partial rule changes can therefore cause many answer regressions even when exact-vector retention appears stable.

The supplied symbolic estimator identifies all languages and executes all held-out targets correctly. No observed nonexact proposal has perfect composition accuracy, although the algebraic +4 counterexample remains valid. Valid saved rules occupy 34 bytes each, including syntax and newline. This compact representation demonstrates externally supplied persistence and execution, not neural retention or a new memory mechanism.

Validity remains an interface limitation. Across all five stages, all 32 invalid 0.5B proposals contain repeated offsets. Of 12 invalid 7B proposals, nine use negative offsets outside the required 0–7 range, two contain malformed JSON, and one repeats an offset. Seed 8012's final checked rule is semantically exact with $-1 \equiv 7 \pmod 8$, but the frozen interface scores it as invalid. We do not normalize scores after collection. Unlike Study C, most proposals satisfy the new interface, but rule correctness remains weak.

Each model uses ten actual generation batches. Whole-suite unpadded input and output token counts are 51,427/4,680 for 0.5B and 50,283/3,305 for 7B. Generation takes 2.724 and 12.457 seconds, with runner totals of 2.763 and 12.497 seconds after loading; peak allocation is 1.269 and 15.546 GiB. Diagnostic checking of all proposals takes 0.00114 and 0.00123 seconds. These costs count shared initial generation once and exclude offline scoring and plotting. Matching revision counts

does not match measured inference cost or the models' pretraining investment.

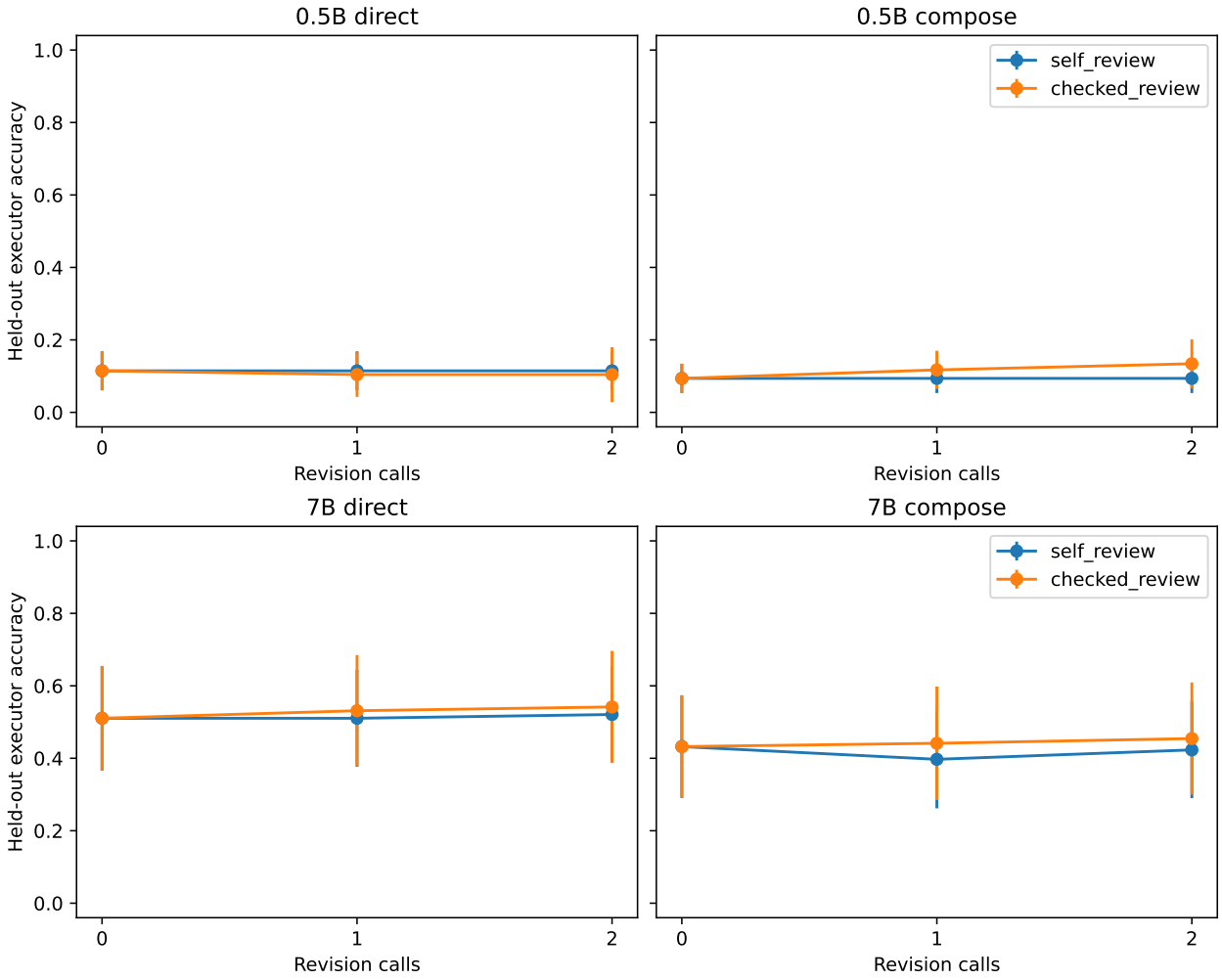


Figure 7: Study F held-out execution from saved rules. Error bars are descriptive 95% t intervals over 24 environments. The two branches share the initial proposal; all revisions run and the final proposal is scored.

6.8 Limitations and interpretation

All six studies use synthetic tasks with disclosed families and inexpensive feedback. They do not measure autonomous exploration, long-horizon reward, human intelligence, or general cross-domain skill accumulation. Study A's fixed optimization schedule may interact with capacity. Its replay treatment also changes fresh-example exposure. Study B uses only one model family and a first-token response interface. Study C adds full generation but also a strict final-line format and a fixed token cap. Study D shows that extending that cap can materially change accuracy with the same prompt and model. Study E tests only 16 online outcomes and reuses the narrow first-token interface; it is not an optimized learning algorithm or a full-generation test. Low accuracy can reflect the interface, instruction-following ability, prior execution ability, or the specific update schedule. Study F adds error-directed rule revision with equal call caps, but neither equal measured compute nor an open-ended reflection agent. Its executor and disclosed rule family contribute supplied

capability, and its final score does not measure learning across languages.

In development, 0.5B answered a simple arithmetic question incorrectly under a one-digit instruction. The official weight SHA-256 matched, and CPU float32, GPU float32/bfloat16, and eager/SDPA paths reproduced it. A subsequent ordinary question produced the correct answer; other ordinary-language responses were coherent. Thus the calibration reveals prompt sensitivity, not general arithmetic incapacity. Model files and device agreement alone cannot certify that a response interface makes full use of the available capability.

The initial Study A random-stream defect and the corrected fresh-seed replication are both disclosed and preserved. A disk-full error occurred only during Study B export; every original run had completion metadata and complete predictions. Export was repeated on a separate filesystem without rerunning models. Study F's first 7B development load failed on a cache metadata write to that full filesystem, before inference; its provenance and traceback are preserved. The same frozen source and cached weights passed after enabling offline cache access. Source hashes and complete outcome grids were checked before analysis.

Prior work already establishes reinforcement learning, external memory, verbal reflection, and test-time computation as relevant mechanisms. Our current evidence is a diagnostic investigation of the stated hypothesis, not a first discovery of feedback-driven intelligence. Targeted review of nine close precedents strengthens the comparison, but a systematic novelty review remains incomplete. Study F tests a small structured external rule, while open-ended memory compression, active exploration, sustained cross-task retention, and long-horizon reward learning remain untested.

Stability is only partially measured by Study A's old-rule retention; repeated rule changes and long-term memory interference remain untested. Our exact synthetic targets do not reproduce the difficulty of obtaining trustworthy real-world feedback. No study tests reward hacking: optimizing a proxy evaluator can improve its score without improving the intended task, particularly if the agent can influence the feedback source. These mechanisms require separate held-out evaluation and feedback-quality experiments; faster updates alone do not address them.

7 Conclusion

Feedback enables substantial adaptation in small neural learners under task change, but capacity remains consequential when both small and large models learn. In the smallest pretrained model, our example-conditioning and limited parameter-update protocols do not establish compositional improvement. Its format failures under Study C's prompt remain unresolved; Study F's new structured interface produces mostly valid rules but no exact 0.5B recovery. In 7B, supplying correct examples yields substantial compositional accuracy once the continuation budget permits most responses to finish. The earlier short-budget failures therefore cannot support a general conclusion that this model cannot learn the disclosed rules in context. An explicit one-pass feedback loop in the 0.5B model yields no demonstrated compositional gain, whether it updates after every outcome or only after an error. Retaining behavior changes therefore cannot be equated with accumulating useful, transferable skill. External rule storage and exact example checks also yield no demonstrated 0.5B composition gain over two self-revisions. The larger model recovers some full rules, while retaining substantial uncertainty and query-level regressions.

The supported conclusion is conditional: feedback, representation, prior learning, update rules, and response interfaces jointly determine measured adaptation. A strong claim about small agents needs evidence of sustained error-directed updates, retention, and transfer across diverse tasks under comparable resource constraints. The present study establishes neither that feedback alone explains intelligence nor that increasing model capacity is an incorrect research direction.

References

[1] D. Silver and R. S. Sutton. Welcome to the Era of Experience. Book-chapter preprint, 2025. Author-linked copy: https://www.davidsilver.uk/publications/.

[2] D. Silver, S. Singh, D. Precup, and R. S. Sutton. Reward is enough. Artificial Intelligence, 299:103535, 2021. https://doi.org/10.1016/j.artint.2021.103535.

[3] A. Vaswani et al. Attention Is All You Need. NeurIPS, 2017. https://arxiv.org/abs/1706.03762.

[4] J. Kaplan et al. Scaling Laws for Neural Language Models. 2020. https://arxiv.org/abs/2001.08361.

[5] J. Hoffmann et al. Training Compute-Optimal Large Language Models. 2022. https://arxiv.org/abs/2203.15556.

[6] H. Chen et al. Feedback is all you need: from ChatGPT to autonomous driving. 2023. https://doi.org/10.1007/s11432-023-3740-x. Bibliographic metadata verified; full-text argument not assessed in this version.

[7] Y. Duan et al. RL^2: Fast Reinforcement Learning via Slow Reinforcement Learning. 2016. https://arxiv.org/abs/1611.02779v2.

[8] M. Laskin et al. In-context Reinforcement Learning with Algorithm Distillation. 2022. https://arxiv.org/abs/2210.14215.

[9] N. Shinn et al. Reflexion: Language Agents with Verbal Reinforcement Learning. 2023. https://arxiv.org/abs/2303.11366v4.

[10] A. Madaan et al. Self-Refine: Iterative Refinement with Self-Feedback. 2023. https://arxiv.org/abs/2303.17651v2.

[11] J. Huang et al. Large Language Models Cannot Self-Correct Reasoning Yet. ICLR, 2024. https://arxiv.org/abs/2310.01798v2.

[12] C. Snell et al. Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters. 2024. https://arxiv.org/abs/2408.03314.

[13] G. Wang et al. Voyager: An Open-Ended Embodied Agent with Large Language Models. 2023. https://arxiv.org/abs/2305.16291v2.

[14] X. Chen et al. Teaching Large Language Models to Self-Debug. 2023. https://arxiv.org/abs/2304.05128v2.

[15] A. Zhao et al. ExpeL: LLM Agents Are Experiential Learners. 2024. https://arxiv.org/abs/2308.10144v3.